

Experiment 2

Title

Designing and Implementing Regular Expressions for Regular Languages

Aim

To design, test, and validate regular expressions for various regular languages using an online regular expression testing tool.

Software and Tools Required

- Regex101 (Online Regex Tester)

Introduction

A **Regular Expression (Regex)** is a sequence of characters that defines a search pattern. Regular Expressions are widely used in Compiler Design during the **Lexical Analysis** phase to identify tokens such as identifiers, keywords, constants, operators, and delimiters.

Instead of checking each character manually, a compiler uses Regular Expressions to efficiently recognize valid patterns in the source program.

For example:

```
int total = 100;
```

The lexical analyzer identifies:

Lexeme	Token
int	Keyword
total	Identifier

= Assignment
 Operator

100 Integer Constant

; Delimiter

Regular Expressions provide the patterns required to identify these tokens automatically.

Relationship Between Regular Expressions and Finite Automata

In the previous experiment, we implemented **Finite Automata**.

Both concepts are closely related:

Regular Expression



Finite Automaton



Lexical Analyzer

Every Regular Expression can be converted into a Finite Automaton, and every Finite Automaton recognizes a Regular Language.

Basic Regular Expression Symbols

Symb ol	Meaning	Example
.	Any single character	a . b
*	Zero or more occurrences	ab*
+	One or more occurrences	ab+
?	Zero or one occurrence	ab?

[]	Character Class	[abc]
[^]	Negated Character Class	[^0-9]
[a-z]	Lowercase letters	[a-z]+
[A-Z]	Uppercase letters	[A-Z]+
[0-9]	Digits	[0-9]+
,	,	OR Operator
()	Grouping	(ab)+
^	Beginning of string	^ abc
\$	End of string	xyz \$

Procedure

1. Open your web browser.
2. Navigate to **Regex101**.
3. Select the default Regular Expression flavor.
4. Enter the Regular Expression in the left panel.
5. Enter sample test strings in the right panel.
6. Observe whether each string matches.
7. Modify the expression if required.
8. Record your observations.

Conclusion

In this experiment, various Regular Expressions were designed and tested using Regex101.

Different categories of strings such as identifiers, numbers, email addresses, mobile numbers, PIN codes, dates, and passwords were successfully recognized. The experiment demonstrated how Regular Expressions can

efficiently describe patterns and how they form the foundation of lexical analysis in compiler design.